



Lab4 实验报告 · RISC-V 目标代码生成

1. 实现思路

本实验基于 Lab 3 生成的 Zero IR，将中间代码翻译为 RISC-V 32 汇编。整体分三个阶段：指令选择、寄存器分配、汇编输出。

指令选择 采用一对一映射模式。InstSelector 遍历每个基本块的 IR 指令，将其转换为对应的 ASM 指令。LoadImm → li，Assign → mv，Binary → 依 op 生成

add/sub/mul/div/rem/slt，Unary 的 Neg 用 sub rd, zero, rs 取负，Lnot 用 sltiu 即 seqz，If → 依条件选 beq/bne/blt/bge，Call/Arg/Param 分别处理寄存器传参和栈传参，Load/Store 处理内存读写，Dec 从 fp 向下分配大对象，全局变量收集后在 emitter 输出 .data 段。由于 IR 操作数既有变量名也有 #k 格式的立即数，处理二元运算时先检测再标准化。

寄存器分配 采用分层策略。先对所有 ASM 指令做活度分析，计算每个虚拟寄存器的 [first, last] 区间，然后分两级分配：

第一级——物理寄存器分配：将热虚拟寄存器（频度高、区间不重叠）直接分配给 callee-saved 物理寄存器（s1-s11），消除 lw/sw 开销。第二级——栈槽贪心复用：未分配到物理寄存器的虚拟寄存器，按活度区间进行贪心槽位分配——两个区间不重叠即可共用同一 sp 偏移。

汇编输出 在 ASMEmitter 中完成。prologue 中根据 func->reg_map 扫描实际使用的 s1-s11 并保存到 temp_stack_size 偏移处，epilogue 中对应恢复。大立即数超 12 位范围时使用 lui + add + lw/sw 分解（比 li 少一条指令）。

2. 难点与亮点

2.1 难点

BinaryOp 与 Arith::Op 的关系运算转换。实验初期尝试 static_cast 直接映射，值域不齐；Gt 需交换操作数，Le/Ge/Eq/Neq 各需两条指令组合。最终用 switch 逐 op 分派。

IR 的 Arg 节点 k 值为 0。IR translator 创建 Arg 时未传 k，多参数函数调用全部进入 a0。解决：在 InstSelector 中增加 arg_index 计数器。

DEC 分配与 outgoing 参数区重叠。两者都从 `sp+0` 分配，outgoing arg 覆盖局部变量地址指针。解决：指令选择前预调用 `alloc_temp(112)` 预留 outgoing 区。

大栈帧立即数溢出。大数组使帧超 4KB，12 位偏移不够。`li` 分解为 4 条指令，优化为 `lui + add + lw/sw` 降至 3 条。进一步将 DEC 大分配从 fp 向下，虚拟寄存器从 sp 向上，使频繁访问的溢留区偏移保持较小。

活度分析与槽复用。朴素方案每个虚拟寄存器独占一个栈槽，far_label 有 1300+ 个虚拟寄存器，溢留区极大。实现线性扫描活度分析——展平指令计算 `[first, last]` 区间，处理循环反向跳转修正区间，贪心复用不重叠的槽位。槽位数从 1300+ 降至 ~200，执行步数从 55711 降至 24621（-56%）。

物理寄存器分配。在活度分析基础上，将不重叠的热虚拟寄存器分配给 callee-saved 寄存器（s1-s11），完全消除这部分寄存器的 lw/sw。emitter 在 prologue/epilogue 中动态保存/恢复实际使用的 s 寄存器。far_label 中反复操作的变量 `a` 被分配到 s1，每次 `a = a + 1` 从 3 条指令 2 次访存变为 1 条指令 0 次访存，最终 53/53 测试全部通过。

2.2 亮点

活度分析 + 物理寄存器分配 + 栈槽贪心复用的三级分配策略；大立即数 `lui` 分解（4→3 指令）；帧布局分离（DEC 高位，虚寄存器低位 + s 寄存器中段）；selectBinary 的 switch 分派体系；多参数栈传参的完整处理（selectArg/selectParam 对 $k < 8$ 和 $k \geq 8$ ）。

3. 心得体会

从 IR 到 RISC-V 汇编的转换让我直观理解了编译器后端的工作流程。最深的体会是：活度分析是寄存器分配的灵魂——只有知道哪些变量可以共享，哪些必须独占，才能做出有效的分配决策。朴素 spill-all 策略虽然简单，但大量的 lw/sw 严重拖慢执行速度；从独占栈槽到贪心复用，再到直接分配物理寄存器，每一层优化都建立在对程序运行时行为的更深入理解之上。调试过程中反复查看 Venus 生成的汇编代码，让我对 RISC-V 的寄存器约定、栈帧布局和调用规范有了具体的认识。

4. 附加内容

借鉴声明

参考了课程提供的 Lab 4 文档和 Zero IR 模板代码 °未复制他人代码 °

AI 工具使用记录

- Proma Agent (Claude Code) :辅助代码审查 `活度分析算法设计` `bug 定位 (selectBinary 的 static\_cast `Arg 的 k=0` DEC 重叠 `函数标签重复 `大立即数分解 `帧布局优化` , 不直接生成实现代码 `由本人独立完成 °
- Gemini :辅助实现 callee-saved 物理寄存器分配及 emitter 中 s 寄存器保存/恢复 °
- 相关 AI 对话记录保留于仓库 `reports/appends/lab4/` 目录 °